NEAR One Robust ECDSA

Security Assessment

February 10, 2026

Prepared for:
Mårten Blankfors
NEAR One

Prepared by: **Paul Bottinelli, Keegan Ryan, and Joe Doyle**

Table of Contents

Table of Contents 1
Project Summary 2
Executive Summary 3
Project Goals 5
Project Targets 6
Project Coverage 7
Automated Testing 8
Summary of Findings 10
Detailed Findings 11
1. ECDSA signature verification does not enforce low s values 11
2. Rerandomization does not perform correct domain separation 13
3. Presignature rerandomization does not provably prevent Wagner's attack 15
4. Signature share linearization may slightly weaken robustness 18
5. Zero threshold causes integer overflow panic in debug mode 20
6. Missing zeroization of presignature data 22
7. Standard split-view attack can extract the secret key with 3t + 2 signers 24
8. Novel split-view attack can extract the secret key with 2t + 3 signers 26
9. Inconsistent interpolation bounds between implementation and reference 28
10. Inconsistent threshold semantics across DKG and presigning protocols 31
A. Vulnerability Categories 33
B. Code Quality Findings 35
C. Automated Analysis Tool Configuration 37
D. Fix Review Results 39
Detailed Fix Review Results 40
E. Fix Review Status Categories 42
About Trail of Bits 43
Notices and Remarks 44

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Paul Bottinelli, Consultant
paul.bottinelli@trailofbits.com

Keegan Ryan, Consultant
keegan.ryan@trailofbits.com

Joe Doyle, Consultant
joseph.doyle@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
October 1, 2025	Pre-project kickoff call
October 7, 2025	Status update meeting #1
October 14, 2025	Status update meeting #2
October 21, 2025	Status update meeting #3
October 29, 2025	Delivery of report draft
October 29, 2025	Report readout meeting
December 1, 2025	Delivery of final comprehensive report
February 3, 2026	Completion of fix review
February 10, 2026	Delivery of report with fix review appendix

Executive Summary

Engagement Overview

NEAR One engaged Trail of Bits to review the security of its honest-majority threshold ECDSA library, `robust_ecdsa`.

This library implements a version of the DJNPO20 threshold ECDSA scheme, with modifications to support additive key derivation, presignature rerandomization, and signing prehashed messages.

A team of three consultants conducted the review from October 6 to October 28, 2025, for a total of six engineer-weeks of effort. Our testing efforts focused on the correctness and security of the presigning and signing implementation. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes. From February 2 to February 3, 2026, one consultant performed a review of the fixes provided by NEAR One, as described in appendix D.

Observations and Impact

We identified two high-severity findings that allow key extraction if signers can disagree on details of the signing session (TOB-NEARROBUST-7, TOB-NEARROBUST-8).

We also identified several implementation flaws, including missing checks in signature verification (TOB-NEARROBUST-1), missing domain separation (TOB-NEARROBUST-6), integer overflow (TOB-NEARROBUST-5), and missing zeroization (TOB-NEARROBUST-6).

Finally, we identified several issues in the design and documentation, including several inconsistencies in the treatment of thresholds and participant indices (TOB-NEARROBUST-4, TOB-NEARROBUST-9, TOB-NEARROBUST-10) and a divergence from the GS21 rerandomization scheme that goes outside the scope of the security proof (TOB-NEARROBUST-3).

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that NEAR One take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Define and document the allowable number of participants in each phase.** The current design and implementation are not secure when signing is performed with more than $2t + 3$ participants (TOB-NEARROBUST-7, TOB-NEARROBUST-8).
- **Develop a security proof.** The protocol deviates from DJNPO20 and GS21 in such a way that the security proofs in each one do not directly apply (TOB-NEARROBUST-3,

TOB-NEARROBUST-8). While we have suggested modifications that mitigate the specific attacks found in this review, they are not a substitute for a security proof.

- **Extend the test suite to cover more edge case behavior.** The current test suite does not detect high s signatures (TOB-NEARROBUST-1), zero thresholds (TOB-NEARROBUST-5), and several potential mistakes in zero-share generation (see Automated Testing).

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	2
Medium	1
Low	1
Informational	6
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Configuration	1
Cryptography	7
Data Exposure	1
Data Validation	1

Project Goals

The engagement was scoped to provide a security assessment of the NEAR One “robust ECDSA” threshold signature component. Specifically, we sought to answer the following non-exhaustive list of questions:

- Is the threshold ECDSA implementation secure and confidential?
- Does the implementation correctly match the documented algorithm?
- How do the specified changes from the DJNPO20 scheme affect the security of the NEAR One implementation?
- Are hash functions used correctly?

Project Targets

The engagement involved reviewing and testing the following target.

threshold-signatures (robust_ecdsa)

Repository	https://github.com/near/threshold-signatures
Version	`657a2e4992ed9759ed5cd248e3004ff823a78783`
Type	Rust
Platform	Native

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Manual review of the robust ECDSA documentation in `docs/ecdsa/robust_ecdsa/signing.md`
    - In particular, we compared the overall specification to the DJNPO20 scheme and the rerandomization approach to the GS21 rerandomization scheme.
- Manual review of the Rust implementation in `src/ecdsa/robust_ecdsa/`
- Manual review of portions of other helper files that are used by the implementation (e.g., `src/crypto/polynomials.rs`)
- Static analysis of the Rust code using the tools detailed in the Automated Testing section

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description
Clippy	An open-source Rust linter used to catch common mistakes and unidiomatic Rust code
Dylint	A tool for running Rust lints from dynamic libraries
`cargo-audit`	An open-source tool for checking dependencies against the RustSec advisory database
`cargo-llvm-cov`	A Cargo plugin for generating LLVM source–based code coverage
`cargo-mutants`	An open-source mutation testing tool for Rust

Areas of Focus

Our automated testing and verification work focused on the following:

- Assessing general code quality
- Identifying gaps in test coverage

Test Results

The results of this focused testing are detailed below.

Clippy
Clippy did not identify any relevant findings in the codebase.

Dylint
Dylint did not identify any relevant findings in the codebase.

cargo-audit
`cargo-audit` identified two unmaintained crates in the dependencies: `paste` and `atomic-polyfill`. We recommend running `cargo audit` on every commit as part of CI.

cargo-llvm-cov

We ran `cargo-llvm-cov` in the `robust_ecdsa` directory and obtained a test code coverage report. The results indicate that there are certain error cases that are not covered by the current test suite, but we did not identify issues with the logic in those cases.

Filename	Function Coverage	Line Coverage	Region Coverage	Branch Coverage
ecdsa/robust_ecdsa/mod.rs	100.00% (2/2)	93.55% (29/31)	91.43% (32/35)	- (0/0)
ecdsa/robust_ecdsa/presign.rs	87.50% (14/16)	85.54% (213/249)	89.67% (382/426)	- (0/0)
ecdsa/robust_ecdsa/sign.rs	100.00% (12/12)	91.77% (212/231)	89.90% (356/396)	- (0/0)
ecdsa/robust_ecdsa/test.rs	100.00% (18/18)	98.21% (220/224)	93.70% (446/476)	- (0/0)

cargo-mutants

We ran `cargo-mutants` in the `robust_ecdsa` folder. This tool mutates the code and determines whether the tests detect this change. The tool identified three modifications to presigning that the test suite does not detect:

- Insufficient blinding, either by setting the blinding values in the presignature to 0 or by using a lower threshold for the zero shares
- An incorrect degree check for R and W
- Participant counts outside the allowable range

To ensure correct behavior, we recommend adding tests that attempt to perform the attacks these implementation details are supposed to prevent.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	ECDSA signature verification does not enforce low s values	Cryptography	Informational
2	Rerandomization does not perform correct domain separation	Cryptography	Informational
3	Presignature rerandomization does not provably prevent Wagner's attack	Cryptography	Informational
4	Signature share linearization may slightly weaken robustness	Cryptography	Low
5	Zero threshold causes integer overflow panic in debug mode	Data Validation	Informational
6	Missing zeroization of presignature data	Data Exposure	Medium
7	Standard split-view attack can extract the secret key with 3t + 2 signers	Cryptography	High
8	Novel split-view attack can extract the secret key with 2t + 3 signers	Cryptography	High
9	Inconsistent interpolation bounds between implementation and reference	Cryptography	Informational
10	Inconsistent threshold semantics across DKG and presigning protocols	Configuration	Informational

Detailed Findings

1. ECDSA signature verification does not enforce low s values

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-NEARROBUST-1
Target: `threshold-signatures/src/ecdsa/mod.rs`	

Description

Since the ECDSA verification equation does not change if the sign of s is flipped, ECDSA verification implementations impose a canonical form for signatures by requiring that s is below half the scalar modulus. The final step of the threshold signing algorithm checks that the generated signature (r, s) passes verification using the `Signature::verify` method, shown in figure 1.1.

```
// This verification tests the signature including whether s has been normalized
pub fn verify(&self, public_key: &AffinePoint, msg_hash: &Scalar) -> bool {
    let r: Scalar = x_coordinate(&self.big_r);
    if r.is_zero().into() || self.s.is_zero().into() {
        return false;
    }
    // tested earlier is not zero, so inversion will not raise an error and unwrap
cannot panic
    let s_inv = self.s.invert_vartime().unwrap();
    let reproduced = (ProjectivePoint::GENERATOR * (*msg_hash * s_inv))
        + (ProjectivePoint::from(*public_key) * (r * s_inv));
    x_coordinate(&reproduced.into()) == r
}
```

Figure 1.1: ECDSA verification; note that the only check done on s is to ensure it is nonzero (`threshold-signatures/src/ecdsa/mod.rs#62–73`)

This check does not enforce the low s requirement. However, as shown in figure 1.2, the s value is normalized in the final step of signing regardless, so this missing validation does not affect the result if the coordinator behaves honestly.

```
// Normalize s
s.conditional_assign(&(-s), s.is_high());

let sig = Signature {
    big_r: presignature.big_r,
    s,
```

```
};

if !sig.verify(&public_key, &msg_hash) {
    return Err(ProtocolError::AssertionFailed(
        "signature failed to verify".to_string(),
    ));
}

Ok(Some(sig))
```

Figure 1.2: The final coordinator step normalizes s before returning the signature. (`threshold-signatures/src/ecdsa/robust_ecdsa/sign.rs#114–128`)

Recommendations

Short term, add low *s* verification to the `Signature::verify` method and add a regression test to confirm it.

Long term, ensure that the signature verification test suite includes all cases that should be rejected.

2. Rerandomization does not perform correct domain separation

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-NEARROBUST-2
Target: `src/ecdsa/mod.rs`	

Description

The rerandomization function uses SHA-3 as a PRNG and generates a nonzero scalar value by rejection sampling. When rejection occurs, the two-byte sequence `[0x00, 0x01]` is appended to the input and a new scalar is generated. This process is shown in figure 2.1.

```
    pub fn derive_randomness(&self) -> Result<Scalar, ProtocolError> {
        // create a string containing (pk, msg_hash, big_r, sorted(participants))
        let pk_encoded_point = self.pk.to_encoded_point(true);
        let encoded_pk: &[u8] = pk_encoded_point.as_bytes();
        let encoded_msg_hash: &[u8] = &self.msg_hash;
        let big_r_encoded_point = self.big_r.to_encoded_point(true);
        let encoded_big_r: &[u8] = big_r_encoded_point.as_bytes();

        // concatenate all the bytes
        let mut concatenation = Vec::new();
        concatenation.extend_from_slice(encoded_pk);
        concatenation.extend_from_slice(encoded_msg_hash);
        concatenation.extend_from_slice(encoded_big_r);
        // Append each ParticipantId's
        for participant in self.participants.participants() {
            concatenation.extend_from_slice(&participant.bytes());
        }

        // initiate hkdf with the salt and with some `good' entropy
        let hk = Hkdf::<sha3::Sha3_256>::new(Some(&Self::SALT), &self.entropy);

        let mut delta = Scalar::ZERO;
        // If the randomness created is 0 then we want to generate a new randomness
        while bool::from(delta.is_zero()) {
            // Generate randomization out of HKDF(entropy, pk, msg_hash, big_r,
participants, nonce)
            // where entropy is a public but unpredictable random string.
            // The nonce is a succession of appended ones of growing length
depending on the number of times
            // we enter into this loop
            let mut okm = [0u8; 32];

            hk.expand(&concatenation, &mut okm)
                .map_err(|_| ProtocolError::HashingError)?;
```

```
            // derive the randomness delta
            delta = Scalar::from_repr(okm.into()).unwrap_or(
                // if delta falls outside the field
                // probability is negligible: in the order of 1/2^224
                Scalar::ZERO,
            );
            // append an extra 0 at the end of the concatenation every time delta is
zero
            concatenation.extend_from_slice(&[0u8, 1]);
        }
        Ok(delta)
    }
}
```

Figure 2.1: Random scalars are rejection-sampled by repeatedly appending to the string that is hashed. (`threshold-signatures/src/ecdsa/mod.rs#127–171`)

This means that the third-try random value from a group with participant list `l` will be the same as the first-try random value from a group with the participant list `l||0x01000100`, since `0x01000100` maps to the byte sequence [0,1,0,1] in little-endian encoding.

This is not exploitable due to the negligible chance of having even two loop iterations, but ensuring unique encodings when passing data to a hash function is good cryptographic practice.

Recommendations

Short term, ensure that the encoding used here is unique; for example, use an incrementing counter appended to the list instead of concatenating a constant.

Long term, ensure that all encoding functions used for passing data into hash functions are injective.

3. Presignature rerandomization does not provably prevent Wagner's attack

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-NEARROBUST-3
Target: `docs/ecdsa/robust_ecdsa/signing.md`	

Description

The robust ECDSA scheme includes a presignature rerandomization step to make the Wagner signature forgery attack infeasible, but the provable guarantees of rerandomization do not extend to the implemented scheme. While this does not mean that the production implementation is insecure, there is a Wagner-style attack against the implementation in test cases that decreases the security from 128 bits to 85 bits.

A number of attacks on ECDSA become possible when an adversary learns the presignature value R used by the final signature before the message hash h and key tweak ϵ are finalized. These attacks and their countermeasures are explored in a 2021 paper by Groth and Shoup (GS21). In this work, they show that when the presignature is rerandomized with a new value δ, the security of ECDSA with presignatures provably reduces to hardness assumptions about the hash function under the elliptic curve generic group model (Theorem 6).

However, there are a number of subtle differences between the proposed and implemented scheme that mean the proof does not apply. First, the presignature is multiplicatively rerandomized ($\delta \cdot R$) rather than additively rerandomized ($R + \delta \cdot G$). More importantly, GS21 assumes δ is chosen by an honest signer in an "unpredictable" way, whereas the implementation generates δ using a hash function over several variables, including a source of public randomness. Since an attacker may observe the public randomness ρ before adversarially choosing h and ϵ, δ is predictable and the proof is invalid. Groth and Shoup speculate that this approach is heuristically secure but caution that this is not formally proven.

Although there is no proven security, it is important that δ depends on the public key X, tweak ϵ, message hash h, and randomness ρ. When this is not the case, the implementation can be vulnerable to a variant of Wagner's attack. While the production code that uses the robust ECDSA implementation correctly follows this advice, the current test cases and documentation do not. These suggest that δ is derived as $\delta \leftarrow HKDF(X, h, R, \rho)$, meaning δ does not depend on ϵ. This setting is vulnerable to Wagner's attack.

Wagner's attack is a generalized collision-finding algorithm that can be applied to reduce the security of ECDSA with nonrandomized presignatures. After the presignature phase, the attacker knows which value of r will appear in the final signature (r, s). The attacker uses this knowledge to find a colliding pair of message hashes (h, h') and public key tweaks (e, e') and obtains a valid signature (r, s) for the innocuous hash h with tweak e. Because of the collision, the attacker can then compute a valid signature (r', s') for the malicious hash h' with tweak e' (in the original attack, $(r', s') = (r, s)$). Finding these four colliding values (h, h', e, e') takes time $2^{n/3}$, which is less expensive than the $2^{n/2}$ cost of finding a collision in the hash function or solving a discrete logarithm.

Presignature rerandomization is meant to prevent this attack by making the final value of r unpredictable and destroying the algebraic relationship that makes finding a collision cheaper than the generic birthday bound. However, such an algebraic relationship remains after the rerandomization in NEAR One's robust ECDSA scheme, and Wagner's attack still applies.

After the presigning phase of this scheme, an adversarial participant learns the value of elliptic curve point R. During the signing phase, every participant receives the public key X, a public source of entropy ρ, a tweak ϵ used during key derivation, and the message hash h. We assume the adversary begins the attack after ρ becomes public and has partial adversarial control over ϵ and h. Using this input, each party computes randomness $\delta \leftarrow HKDF(X,\ h,\ R,\ \rho)$. Under this setting, the attacker can repeatedly sample candidate messages offline and determine the resulting values of h and δ; these values are uniformly distributed. Similarly, we assume the tweak ϵ is determined by a hash function and independent of the choice of message.

After the signing phase, the coordinator reveals the signature $(\delta \cdot R, s)$, where

$$
\begin{aligned}
R &= k \cdot G \\
r &= (\delta \cdot R)_x \\
s &= k^{-1}\delta^{-1}(h + r(x + \epsilon)).
\end{aligned}
$$

This is a valid signature for message hash h and tweaked public key $X + \epsilon \cdot G$.

In our modified version of the Wagner attack, the attacker generates a candidate list of innocuous hashes h for the signature request, a list of malicious hashes h' for the forgery, a list of tweaks ϵ for the request, and a list of tweaks ϵ' for the forgery. Note that the choice of h (or h') influences the value of δ and r (δ' and r', respectively) in a predictable way. The attacker wishes to find a collision of the form $r^{-1}h + \epsilon = r'^{-1}h' + \epsilon'$. Since each term of this relation can be sampled independently and uniformly by the attacker, this is an instance of the 4-sum problem. Wagner's algorithm for the generalized birthday problem finds such a relation in time $2^{n/3}$ and space $2^{n/3}$.

The attacker obtains a signature $(\delta \cdot R, s)$ on the innocuous message with tweak ϵ and uses it to forge a signature $(\delta' \cdot R,\ s')$ on the malicious message with tweak ϵ' by computing

$$\begin{aligned} s' &= \delta\delta'^{-1} r^{-1} r' s \\ &= \delta\delta'^{-1} r^{-1} r' k^{-1} \delta^{-1} (h + r(x + \epsilon)) \\ &= \delta'^{-1} r' k^{-1} (r^{-1} h + x + \epsilon) \\ &= \delta'^{-1} r' k^{-1} (r'^{-1} h' + x + \epsilon') \\ &= \delta'^{-1} k^{-1} (h' + r'(x + \epsilon')). \end{aligned}$$

This forgery is a valid signature, and the attacker has succeeded.

Recommendations

Short term, document the importance of including the tweak ϵ when deriving δ, and update the test cases or the function signatures to enforce secure usage of the library. For example, δ may be derived using $\delta \leftarrow HKDF(X,\ \epsilon,\ h,\ R,\ \rho)$.

While it may be equally secure to replace tuple $(X,\ \epsilon)$ with tweaked key $X + \epsilon \cdot G$, the former approach is more defensive and may be easier to analyze. These approaches are not proven to be secure, but they prevent the attack described above.

Long term, pursue a modified GS21 proof that includes the multiplicative rerandomization and predictable δ. Alternatively, it may be possible to tweak the signing phase to use additive rerandomization with a collaboratively generated and unpredictable δ, in which case the GS21 proof would apply, but this may degrade the performance of the signing phase.

4. Signature share linearization may slightly weaken robustness

Severity: **Low**	Difficulty: **Medium**
Type: Cryptography	Finding ID: TOB-NEARROBUST-4
Target: `src/ecdsa/robust_ecdsa/sign.rs`	

Description

The robust ECDSA implementation introduces several optimizations not included in the original paper (DJNPO20), one of which can allow a single malicious party to prevent the signing phase from completing, even though the original scheme would output a correct signature in the same context. Specifically, a sign phase computation optimization has the participants linearize their signature shares before sending them to the coordinator (see the code excerpt in figure 4.1), which in turns simply sums up the received elements to compute the final s portion of the signature.

```
let linearized_s_me = s_me * participants.lagrange::<C>(me)?;
```

Figure 4.1: `threshold-signatures/src/ecdsa/robust_ecdsa/sign.rs#L146-L146`

This modification introduces a subtle difference compared to the original scheme. Namely, the participants are now required to know the complete signing party set up front, and each of the participants has to behave honestly. Indeed, a single incorrect share would render the final signature incorrect. While this last fact is not specific to the modified scheme under review, we describe a scenario where a malicious participant is able to negatively impact the result of the signature process, which they would not be able to do in the original scheme.

Recall that in the original scheme, the participants interpolate the value of s from $2t + 1$ contributions (see figure 4.2). This means that if a malicious participant sends an incorrect share at position $2t + 2$, a correct signature would still be produced.

P_i on $(\texttt{SIGN-R1}, s_i)$ from all P_j:
  Let $s = \texttt{Int2t}(s_1, s_2, \ldots, s_{2t+1})$
  If $s = 0$: Output $(\texttt{ABORT})$ to E and halt [Check8]
  If $R^s \neq g^m y^r$: Output $(\texttt{ABORT})$ to E and halt [Check9]
  Output $(\texttt{SIGN-END}, r, s)$ to E

Figure 4.2: Excerpt from DJNPO20 with final interpolation step highlighted

In contrast, since contributions from all participants are used by the coordinator to create the signature in the modified scheme, an incorrect share at position $2t + 2$ would induce a failure of the signature generation process. Note that the original scheme does not make any claims as to the robustness of the scheme, at least for the common definition of robustness stating that a correct signature is computed even if up to t players behave maliciously during the signature protocol. Thus, the severity of this slight reduction in robustness in the modified protocol seems to be low.

Another interesting side effect of the linearization optimization is that an arbitrary number of malicious parties can provide contributions in the signing phase (without having participated in the presigning phase) as long as the sum of their contributions is equal to 0.

Exploit Scenario

A distributed system deploys this robust ECDSA implementation with n participants and threshold t, such that $n > 2t + 1$. During a signing operation, $2t + 2$ participants respond promptly with their signature shares, meeting the theoretical requirement of $2t + 1$. However, an attacker (or a legitimate party experiencing network delays) does not send their share. The coordinator cannot complete the signature because it waits for all participants to respond. The signing operation fails or times out, despite having sufficient shares to complete the protocol.

Recommendations

Short term, consider whether the slight reduction in robustness described above is significant enough that the protocol should be reverted to the original coordinator-based interpolation.

Long term, consider modifying the implementation to perform linearization after receiving $2t + 1$ shares from responding participants as in the original protocol. Document the protocol's participant coordination requirements and consider implementing adaptive signing strategies. Add configuration options that allow deployments to choose between waiting for all participants (when high availability is guaranteed) and accepting the first $2t + 1$ responders (when network conditions are uncertain).

5. Zero threshold causes integer overflow panic in debug mode

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Validation	Finding ID: TOB-NEARROBUST-5
Target: `src/dkg.rs`	

Description

While technically outside of the review scope, the distributed key generation (DKG) protocol implementation does not validate that the threshold parameter is at least one before performing arithmetic operations that assume a nonzero threshold. When the threshold is set to zero, the code attempts to compute `threshold - 1` using unsigned integer arithmetic (`usize`), which causes an integer overflow panic in debug builds.

The vulnerable computation occurs in the `do_keyshare` function when generating the secret polynomial:

```
let secret_coefficients =
    Polynomial::<C>::generate_polynomial(Some(secret), threshold - 1, rng)?;
```

Figure 5.1: `threshold-signatures/src/dkg.rs#L358–L359`

The `assert_keygen_invariants` function validates that the threshold does not exceed the number of participants but does not check for a minimum threshold value. A threshold of zero is arguably meaningless in a threshold signature scheme; it essentially means that any party can produce a signature.

Note that this code path can be triggered from within the robust ECDSA test suite, which is why this fragile out-of-scope code portion was identified.

Exploit Scenario

A developer integrating the threshold ECDSA library into their application misunderstands the threshold parameter semantics and sets it to zero. During testing with debug assertions enabled, the application panics when calling the DKG protocol, terminating the application unexpectedly.

Recommendations

Short term, add validation in the `assert_keygen_invariants` function to reject threshold values less than one. Before the existing threshold comparison, add a check that returns an error if the threshold is zero. The same validation should be added to `reshare_assertions` and any other functions that accept threshold parameters.

Long term, consider defining a new `Threshold` type that encodes the minimum value constraint in the type system, preventing construction of invalid threshold values. Add property-based tests that verify that all threshold-related functions correctly handle boundary conditions including zero, one, and maximum values.

6. Missing zeroization of presignature data

Severity: **Medium**	Difficulty: **High**
Type: Data Exposure	Finding ID: TOB-NEARROBUST-6
Target: `src/ecdsa/robust_ecdsa/mod.rs`	

Description

The `PresignOutput` and `RerandomizedPresignOutput` structures contain secret presignature shares that remain in memory after use because the structures do not implement automatic memory zeroization. If these presignature shares are reused across multiple signatures, an attacker could recover the private key share through standard ECDSA nonce reuse attacks.

Figure 6.1 shows that the `PresignOutput` structure is defined without any zeroization mechanism:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PresignOutput {
    /// The public nonce commitment.
    pub big_r: AffinePoint,

    /// Our secret shares of the nonces.
    pub c: Scalar,
    pub e: Scalar,
    pub alpha: Scalar,
    pub beta: Scalar,
}

/// The output of the presigning protocol.
```

Figure 6.1: `PresignOutput` structure without zeroization (`threshold-signatures/src/ecdsa/robust_ecdsa/mod.rs#L24–L36`)

Figure 6.2 shows the same for `RerandomizedPresignOutput`:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RerandomizedPresignOutput {
    /// The rerandomized public nonce commitment.
    big_r: AffinePoint,

    /// Our rerandomized secret shares of the nonces.
    e: Scalar,
    alpha: Scalar,
```

```
    beta: Scalar,
}
```

Figure 6.2: `RerandomizedPresignOutput` structure without zeroization (`threshold-signatures/src/ecdsa/robust_ecdsa/mod.rs#L39–L48`)

Although the `zeroize` crate (version 1.8.1) is available in the project dependencies, neither structure derives or implements the `Zeroize` or `ZeroizeOnDrop` traits. The underlying `Scalar` type from `k256` implements `DefaultIsZeroes` but does not automatically zeroize dropped values. Without explicit zeroization, presignature data persists in memory until the pages are reused, potentially surviving for extended periods in heap memory.

Exploit Scenario

An application uses the robust ECDSA implementation to sign transactions. A memory disclosure vulnerability exists elsewhere in the application, allowing an attacker to read portions of the process memory. After a signing operation completes, the presignature values remain in memory because they are not zeroized. The attacker exploits the memory disclosure to extract a participant's presignature data, specifically the `c` and `beta` values. The attacker can recover the victim's private key share `x` since $\beta = c \cdot x$.

Recommendations

Short term, implement automatic memory zeroization for both presignature structures by having them derive the `ZeroizeOnDrop` trait.

Long term, consider wrapping presignature data in a secure container type that enforces single-use semantics through the type system, making it impossible to reuse a presignature even if the zeroization is bypassed. Additionally, conduct a comprehensive audit of all cryptographic material in the codebase to ensure consistent application of zeroization patterns, and establish coding standards that require `ZeroizeOnDrop` for all structures containing secret values.

7. Standard split-view attack can extract the secret key with 3t + 2 signers

Severity: **High**	Difficulty: **Medium**
Type: Cryptography	Finding ID: TOB-NEARROBUST-7
Target: `docs/ecdsa/robust_ecdsa/signing.md`	

Description

The robust ECDSA scheme is vulnerable to a key extraction attack when honest parties have a different view of the message or the tweak in the signing phase. When different subsets of parties use shares of the same presignature but disagree on what is being signed, each subset can produce a valid signature with multiplicatively related nonces. The private key can be recovered by performing a simple variant of the standard ECDSA nonce reuse attack.

In the robust ECDSA scheme, the presigning phase implicitly determines a nonce k to use in the signing phase with private key x. The signing phase takes a hash h_i and public key tweak ϵ_i as input, using subscripts to denote potentially split views of the signing parameters. These are used to determine a rerandomization value δ_i, and if there are at least $2t + 1$ participants that agree on (h_i, ϵ_i), the signing phase outputs a signature (r_i, s_i) satisfying $s_i = k^{-1}\delta_i^{-1}(h_i + r_i(x + \epsilon_i))$.

The key extraction attack involves two subsets of participants, each of size $2t + 1$, and the first and second subset produce valid signatures for (h_1, ϵ_1) and (h_2, ϵ_2), respectively. This requires $4t + 2$ honest parties that use presignatures only once, or $3t + 2$ total parties if t parties are malicious and participate in both signing phases.

Each of the two signatures results in a linear equation relating x and k, and the private key can be computed using basic algebra.

$$
\begin{aligned}
k &= (s_1^{-1}\delta_1^{-1}h_1 + s_1^{-1}\delta_1^{-1}r_1\epsilon_1) + (s_1^{-1}\delta_1^{-1}r_1)x \\
k &= (s_2^{-1}\delta_2^{-1}h_2 + s_2^{-1}\delta_2^{-1}r_2\epsilon_2) + (s_2^{-1}\delta_2^{-1}r_2)x \\
\Rightarrow \quad x &= (s_1^{-1}\delta_1^{-1}(h_1 + r_1\epsilon_1) - s_2^{-1}\delta_2^{-1}(h_2 + r_2\epsilon_2))(s_2^{-1}\delta_2^{-1}r_2 - s_1^{-1}\delta_1^{-1}r_1)^{-1}
\end{aligned}
$$

This is valid as long as $s_2^{-1}\delta_2^{-1}r_2 \neq s_1^{-1}\delta_1^{-1}r_1$, which happens when $h_1 \neq h_2$ or $\epsilon_1 \neq \epsilon_2$.

Exploit Scenario

$2t + 2$ honest parties and t malicious parties (or $4t + 2$ honest parties) participate in the presigning phase. The attacker initiates one signing phase with $t + 1$ honest parties and all malicious parties (or $2t + 1$ honest parties) and obtains a signature for (h_1, ϵ_1). The attacker initiates a second phase with the remaining honest parties and obtains a signature for $(h_2, \epsilon_2) \neq (h_1, \epsilon_1)$. Each honest party uses their presignature once and is unaware of the signing phase they are not involved in. The attacker computes the secret key and can forge signatures without involving the honest parties.

Recommendations

Short term, ensure that this is not exploitable in the production code. This can be done by ensuring that exactly $N_1 = 2t + 1$ parties are involved in presigning, preventing split-view attacks during signing. Document this threat publicly so users of this library do not call it in a vulnerable way. As is discussed in TOB-NEARROBUST-8, it is not safe to allow $2t + 1 < N_1 < 3t + 2$.

Long term, consider additional communication rounds to ensure that all N_1 parties agree on the subset of size N_2 involved in signing and all N_2 signers agree on the message hash h and tweak ϵ. Build these rounds into the library to prevent accidental misuse.

8. Novel split-view attack can extract the secret key with 2t + 3 signers

Severity: **High**	Difficulty: **Medium**
Type: Cryptography	Finding ID: TOB-NEARROBUST-8
Target: `docs/ecdsa/robust_ecdsa/signing.md`	

Description

When threshold ECDSA is implemented with support for presignatures, it is critical that each presignature can be used only once to prevent the creation of two signatures with the same nonce. However, when the number of signing participants is below the number of presigning participants, malicious participants can cause a presignature to be used twice without any honest participant using their own presignature share twice.

Denote the number of presigning participants as N_1, the number of signing participants as N_2, and the number of malicious participants as t. Any threshold scheme with presignatures is potentially vulnerable to this if $2N_2 \leq N_1 + t$, since two signing sessions can be done in parallel with $N_2 - t$ honest participants in each session and the t malicious participants acting in both. If the two sessions use different messages, the resulting signatures will have the same nonce and, therefore, leak the secret key.

However, NEAR One's variant of the DJNPO20 scheme is vulnerable to a related split-view attack that does not require each view to successfully create a signature. Suppose that the attacker controls the coordinator and t malicious participants, and can convince $t + 1$ honest parties to participate in one signing session and two other honest parties to participate in a separate signing session using the same presignature.

For simplicity, assume that the participants in one signing session have indices 1 and 2, the malicious participants have indices 3 through $t + 3$, and the remaining honest participants in have indices $t + 4$ through $2t + 3$. The coordinator receives linearized versions of $s_1, \ldots, s_{2t+3}$, where for $i = 1, 2$, $s_i = \alpha_i \cdot \delta_1^{-1} \cdot h_1 + (c_i x_i + c_i \epsilon_1) \cdot (\delta_1 R)_x + e_i$ and for the others, $s_i = \alpha_i \cdot \delta_2^{-1} \cdot h_2 + (c_i x_i + c_i \epsilon_2) \cdot (\delta_2 R)_x + e_i$. The attacker has full control of (ϵ, h) for each view, and we assume that $\delta_1 \neq \delta_2$, but otherwise δ is unrestricted.

The attacker divides the linearized signature shares by the corresponding Lagrange polynomial evaluation to recover each s_i, and, using shares 3 through $2t + 3$, computes the degree-$2t$ polynomial $S_2(Y) = \alpha(Y) \cdot \delta_2^{-1} \cdot h_2 + (c(Y) \cdot x(Y) + c(Y)\epsilon_2) \cdot (\delta_2 R)_x + e(Y)$. Based on this polynomial, the attacker can compute $S_2(1)$ and $S_2(2)$, which are the

signature shares participants 1 and 2 *would have sent* if they were participating in the other view. Thus, for $i = 1, 2$,

$$s_i - S(i) = \alpha_i \cdot (\delta_1^{-1} \cdot h_1 - \delta_2^{-1} \cdot h_2) + (c_i x_i + c(i)\epsilon_1) \cdot (\delta_1 R)_x - (c_i x_i + c(i)\epsilon_2) \cdot (\delta_2 R)_x + (e_i - e_i)$$

Setting $h_1 = h_2 = 0$ and noting that the e_i terms cancel, the attacker learns $s_i - S(i) = (c(i)x(i) + c(i)\epsilon_1)(\delta_1 R)_x - (c(i)x(i) + c(i)\epsilon_2)(\delta_2 R)_x$. The polynomials $c(Y)$ and $x(Y)$ are only degree-t, so the attacker can write them symbolically in terms of only one unknown each; that is, if $L_k(Y)$ is the kth degree-t Lagrange polynomial for k-coordinates in the range $k = 2,..., t + 2$, then $c(Y) = c_{known}(Y) + c_2 L_2(Y)$ and $x(Y) = x_{known}(Y) + x_2 L_2(Y)$, where $c_{known}(Y) = \sum_{k=3}^{t+3} c_k L_k(Y)$ and $x_{known}(Y) = \sum_{k=3}^{t+3} x_k L_k(Y)$ are computed only from the presignature and key shares of the malicious participants. For each $i = 1, 2$, this yields one equation,

$s_i - S(i) = A + Bc_2 + Cx_2 + Dc_2 x_2$, where the coefficients A, B, C, D can be calculated from known values. Through Gaussian elimination, one can get a linear equation relating x_2 and c_2, which can be substituted into the other equation to get a quadratic equation for x_2, at which point the attacker simply checks which of the two solutions matches the public key.

Exploit Scenario

Alice has deployed the NEAR One robust ECDSA signature scheme, and because she is aware of the typical split-view attack, she restricts the number of participants in any signing or presigning session to be at most $3t$. However, Bob is able to initiate a split-view attack against a presignature involving $2t + 3$ participants; he uses the techniques described above to extract the secret key.

Recommendations

Short term, do not allow the 0 message hash to be signed. In addition, consider requiring that the number of participants in a presigning or signing session is exactly $2t + 1$. Document that creating more presignature shares could lead to this attack and to a leak of the secret key.

Long term, develop a security proof for the modified DJNPO20 scheme. This attack uses several of the modifications made in the NEAR One design, but we have not determined whether removing any particular modification would lead to a secure design.

9. Inconsistent interpolation bounds between implementation and reference

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Cryptography	Finding ID: TOB-NEARROBUST-9
Target: `src/ecdsa/robust_ecdsa/presign.rs`, `docs/ecdsa/robust_ecdsa/signing.md`	

Description

This finding describes two slight inconsistencies in which implementation and reference diverge regarding participant set sizes and polynomial interpolation degrees.

First, the reference documentation states that the signing phase requires a participant set of size $N_2 > t$ (strictly greater than the threshold).

```
# Signing

In this phase, a set of parties $\mathcal{P}_2 \subseteq \mathcal{P}_1$
of size $N_2 > t$ wishes to generate an ECDSA signature.
```

Figure 9.1: `threshold-signatures/docs/ecdsa/robust_ecdsa/signing.md?plain=1#L72–L75`

However, the size of the participant set in the signing phase must be $N_2 \geq 2t + 1$, since the signature shares s_i correspond to a degree $2t$ polynomial. Thus, fewer than $2t + 1$ shares are not enough to correctly reconstruct the signature. In practice, the implementation performs this interpolation using shares from *all* participants.

```
while !seen.full() {
    let (from, s_i): (_, SerializableScalar<C>) = chan.recv(wait_round).await?;
    if !seen.put(from) {
        continue;
    }
    // Sum the linearized shares
    s += s_i.0;
}
```

Figure 9.2: `threshold-signatures/src/ecdsa/robust_ecdsa/sign.rs#L99–L106`

A related observation is that the signing phase does not validate whether it has sufficient presignature inputs to correctly terminate. The `sign` function accepts a participant list as a parameter and expects contributions from all parties, but it does not ensure that the list is larger than the required $2t + 1$ bound.

The second slight inconsistency concerns the polynomial interpolation for computing the shared value w. The implementation performs interpolation using all N_1 participants while the reference specification requires polynomial interpolation of degree $2t$, see line 64 of the reference, which states the following:

```
11. Each $P_i$ performs polynomial interpolation of degree $2t$ to derive $w$ as in
$w \gets \sum_i \lambda(\mathcal{P}_1)_i \cdot w_i$.
```

Figure 9.3: `threshold-signatures/docs/ecdsa/robust_ecdsa/signing.md?plain=1#L64`

The degree $2t$ requirement corresponds to using exactly $2t + 1$ participants for interpolation. However, the implementation uses all participants, as can be seen in the excerpt in figure 9.4:

```rust
let identifiers: Vec<Scalar> = signingshares_map
    .participants()
    .iter()
    .map(crate::protocol::Participant::scalar::<C>)
    .collect();

let signingshares = signingshares_map
    .into_vec_or_none()
    .ok_or(ProtocolError::InvalidInterpolationArguments)?;

// ...

// polynomial interpolation of w
let w = Polynomial::eval_interpolation(&identifiers, &signingshares, None)?;
```

Figure 9.4: Implementation using all N_1 participants for interpolation (`src/ecdsa/robust_ecdsa/presign.rs#L161–L216`*)*

The code collects identifiers and signing shares from all participants in the `signingshares_map` (which is populated with all N_1 participants), and then performs interpolation using the complete set.

This slight discrepancy does not seem to have any serious consequences, as an incorrectly interpolated w value would get caught by subsequent checks.

Recommendations

Short term, update the reference documentation to state that the signing phase requires a participant set of size $N_2 \geq 2t + 1$. Note that section 3 of the reference paper (DJNPO20) describes an updated protocol using a verifiable secret sharing (VSS) scheme that allows a smaller participant set to recompute the final signature. Consider adding the threshold parameter to the `sign` function signature and adding validation to ensure that the number of participants meets the required bound before the signature computation can continue.

Additionally, either update step 11 of the reference specification to state that polynomial interpolation of degree $N_1 - 1$ be performed, or update the implementation to use only $2t + 1$ participants.

Long term, create test cases that validate the bounds checking and interpolation behavior against the documented protocol requirements. Add comments in the code explaining any intentional deviations from the reference specification and the security rationale behind them.

10. Inconsistent threshold semantics across DKG and presigning protocols

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Configuration	Finding ID: TOB-NEARROBUST-10
Target: `src/ecdsa/robust_ecdsa/presign.rs`, `src/ecdsa/robust_ecdsa/test.rs`	

Description

The threshold parameter has inconsistent semantic meanings across the DKG and the robust ECDSA implementation. In the DKG protocol, the *threshold* represents the reconstruction bound—that is, the minimum number of participants required to reconstruct the secret. However, in the robust ECDSA protocol, the threshold parameter represents the maximum number of malicious parties directly, as explicitly documented in the warning preceding the `do_presign` function:

```
/// /!\ Warning: the threshold in this scheme is the exactly the
///              same as the max number of malicious parties.
#[allow(clippy::too_many_lines)]
async fn do_presign(
    mut chan: SharedChannel,
    participants: ParticipantList,
    me: Participant,
    args: PresignArguments,
    mut rng: impl CryptoRngCore,
) -> Result<PresignOutput, ProtocolError> {
    let rng = &mut rng;
    let threshold = args.threshold;
    // Round 0

    let polynomials = [
        // degree t random secret shares where t is the max number of malicious
parties
        Polynomial::generate_polynomial(None, threshold, rng)?, // fk
        Polynomial::generate_polynomial(None, threshold, rng)?, // fa
        // degree 2t zero secret shares where t is the max number of malicious
parties
        zero_secret_polynomial(2 * threshold, rng)?, // fb
        zero_secret_polynomial(2 * threshold, rng)?, // fd
        zero_secret_polynomial(2 * threshold, rng)?, // fe
    ];
```

Figure 10.1: Presigning protocol with threshold as max malicious parties (`threshold-signatures/src/ecdsa/robust_ecdsa/presign.rs#L81–L103`)

In contrast, the DKG protocol uses the convention where the threshold parameter represents the minimum number of participants needed for reconstruction:

```
async fn do_keyshare<C: Ciphersuite>(
    mut chan: SharedChannel,
    participants: ParticipantList,
    me: Participant,
    threshold: usize,
    secret: Scalar<C>,
    old_reshare_package: Option<(VerifyingKey<C>, ParticipantList)>,
    rng: &mut impl CryptoRngCore,
) -> Result<KeygenOutput<C>, ProtocolError> {
    let mut all_full_commitments = ParticipantMap::new(&participants);
    let mut domain_separator = 0;

    // ...

    // the degree of the polynomial is threshold - 1
    let secret_coefficients =
        Polynomial::<C>::generate_polynomial(Some(secret), threshold - 1, rng)?;
```

Figure 10.2: DKG protocol with threshold as minimum participants for reconstruction (`threshold-signatures/src/dkg.rs#L330–L359`)

While this inconsistency seems to be known to the NEAR One developers, it creates confusion and increases the risk of misconfiguration; for example, future users of the library might use incorrect threshold values across the different protocols.

Recommendations

Short term, standardize the threshold semantics across all protocols in the library and add documentation to clearly indicate the convention used.

Long term, consider introducing type-safe threshold parameters through distinct types such as `ReconstructionThreshold` and `MaxMaliciousParties` that cannot be confused or used interchangeably. Add compile-time checks or runtime assertions that verify that the relationship between DKG and presigning thresholds matches the security requirements, failing early if mismatched parameters are detected.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Findings

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **The specification contains typos.** Step 1 refers to "i" where it should refer to "j", and steps 10 and 11 are in the incorrect order.

```
1. $\bullet$ Each $P_i$ waits to receive $(R_i, w_i)$ from each other $P_j$.
2. $\blacktriangle$ Each $P_i$ *asserts* that:
$\forall j \in \\{t+2.. n\\},\quad \mathsf{ExponentInterpolation}(R_1, \ldots
R_{t+1}; j) =  R_j$
3. Each $P_i$ computes $R \gets \mathsf{ExponentInterpolation}(R_1, \ldots R_{t+1};
0)$
4. $\blacktriangle$ Each $P_i$ *asserts* that $R \neq Identity$
5. Each $P_i$ computes $W_i \gets R^{a_i}$
6. $\star$ Each $P_i$ sends $W_i$ to every other party.
7. $\bullet$ Each $P_i$ waits to receive $W_j$ from every other party.
8. $\blacktriangle$ Each $P_i$ *asserts* that:
$\forall j \in \\{t+2.. n\\},\quad \mathsf{ExponentInterpolation}(W_1, \ldots
W_{t+1}; j) =  W_j$
9. Each $P_i$ computes $W \gets \mathsf{ExponentInterpolation}(W_1, \ldots W_{t+1};
0)$
10. $\blacktriangle$ Each $P_i$ *asserts* that $W = w\cdot G$
11. Each $P_i$ performs polynomial interpolation of degree $2t$ to derive $w$ as in
$w \gets \sum_i \lambda(\mathcal{P}_1)_i \cdot w_i$.
12. $\blacktriangle$ Each $P_i$ *asserts* that $w \neq 0$.
13. Each $P_i$ computes $c_i \gets a_i \cdot w^{-1}$
14. Each $P_i$ computes $\alpha_i \gets c_i+d_i$
15. Each $P_i$ computes $\beta_i \gets c_i \cdot x_i$.
```

Figure B.1: `threshold-signatures/docs/ecdsa/robust_ecdsa/signing.md#L52–L68`

- **Signatures include R instead of just its x-coordinate.** ECDSA signatures only contain the x-coordinate of the point R, but the `Signature` struct contains the point R itself. This appears to be intentional, but it may lead to malleability problems, counteracting the canonicalization provided by reduced s signatures.

```rust
pub struct Signature {
    /// This is the entire first point.
    pub big_r: AffinePoint,
    /// This is the second scalar, normalized to be in the lower range.
    pub s: Scalar,
}
```

Figure B.2: `threshold-signatures/src/ecdsa/mod.rs#L54–L59`

- **Presigning will fail if k_i or a_i is 0.** Serialization of coefficient commitments is performed via the frost-secp256k1 implementation of the `<Secp256K1Group as Group>::serialize` function, shown in figure C.3, which returns an error on the identity element. This happens with only negligible probability.

```
fn serialize(element: &Self::Element) -> Result<Self::Serialization, GroupError> {
    if *element == Self::identity() {
        return Err(GroupError::InvalidIdentityElement);
    }
    let mut fixed_serialized = [0; 33];
    let serialized_point = element.to_affine().to_encoded_point(true);
    let serialized = serialized_point.as_bytes();
    fixed_serialized.copy_from_slice(serialized);
    Ok(fixed_serialized)
}
```

Figure B.3: `frost/frost-secp256k1/src/lib.rs#L120–L129`

C. Automated Analysis Tool Configuration

This appendix describes the setup of the automated analysis tools used during this audit.

Though static analysis tools frequently report false positives, they detect certain categories of issues, such as memory leaks, misspecified format strings, and the use of unsafe APIs, with essentially perfect precision. We recommend periodically running these static analysis tools and reviewing their findings.

We convert tool output to the SARIF file format (e.g., with `clippy-sarif`) when possible, allowing for easy inspection of the results within an IDE (e.g., using VS Code's SARIF Explorer extension).

Clippy

The Rust linter Clippy can be installed using `rustup` by running the command `rustup component add clippy`. Invoking `cargo clippy -- -W clippy::pedantic` in the root directory of the project runs the tool with the pedantic ruleset.

Clippy results from the regular set of rules should always be fixed.

```
# run clippy in regular and pedantic mode and output to SARIF
cargo clippy --message-format=json | clippy-sarif > clippy.sarif
cargo clippy --message-format=json -- -W clippy::pedantic -Wclippy::or_fun_call |
clippy-sarif > clippy_pedantic.sarif
```

Figure C.1: The invocation commands used to run Clippy on the codebase

Dylint

Dylint is a tool for running Rust lints from dynamic libraries, similar to Clippy. We ran the general and supplementary rulesets against the codebase.

```
cargo dylint --no-deps --git https://github.com/trailofbits/dylint --pattern
examples/general -- --message-format=json | clippy-sarif > general.sarif

cargo dylint --no-deps --git https://github.com/trailofbits/dylint --pattern
examples/supplementary -- --message-format=json | clippy-sarif > supplementary.sarif
```

Figure C.2: The command used to run Dylint on the project

test-fuzz

The `test-fuzz` Cargo plugin was used to quickly generate a fuzzing corpus and to implement a fuzzing harness for the safe serialization routines.

To install it, run the following command:

```
cargo install cargo-test-fuzz cargo-afl
```

Figure C.3: The invocations used to install `cargo-test-fuzz` and its dependencies

cargo-audit

The `cargo-audit` Cargo plugin identifies known vulnerable dependencies in Rust projects. It can be installed using `cargo install cargo-audit`. To run the tool, run `cargo audit` in the crate root directory.

cargo-llvm-cov

The `cargo-llvm-cov` Cargo plugin is used to generate LLVM source-based code coverage data. The plugin can be installed via the command `cargo install cargo-llvm-cov`. To run the plugin, run the `cargo llvm-cov` command in the crate under test.

cargo-mutants

The `cargo-mutants` Cargo plugin repeatedly mutates the codebase and then runs the test suite to identify under-tested code paths in the codebase. The plugin can be installed via the command `cargo install cargo-mutants`. To run the plugin, run the `cargo mutants` command in the crate under test.

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From February 2 to February 3, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the NEAR One team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, NEAR One has resolved all of the 10 issues described in this report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	ECDSA signature verification does not enforce low s values	Informational	Resolved
2	Rerandomization does not perform correct domain separation	Informational	Resolved
3	Presignature rerandomization does not provably prevent Wagner's attack	Informational	Resolved
4	Signature share linearization may slightly weaken robustness	Low	Resolved
5	Zero threshold causes integer overflow panic in debug mode	Informational	Resolved
6	Missing zeroization of presignature data	Medium	Resolved
7	Standard split-view attack can extract the secret key with 3t + 2 signers	High	Resolved
8	Novel split-view attack can extract the secret key with 2t + 3 signers	High	Resolved
9	Inconsistent interpolation bounds between implementation and reference	Informational	Resolved

10	Inconsistent threshold semantics across DKG and presigning protocols	Informational	Resolved

Detailed Fix Review Results

TOB-NEARROBUST-1: ECDSA signature verification does not enforce low s values
Resolved in PR #154. The signature verification now returns false if s is high.

TOB-NEARROBUST-2: Rerandomization does not perform correct domain separation
Resolved in PR #154. Instead of appending a value that could collide with user-supplied data, the rerandomization implementation now prepends a counter.

TOB-NEARROBUST-3: Presignature rerandomization does not provably prevent Wagner's attack
Resolved in PR #180. The documentation, test cases, and API now explicitly include the tweak when deriving presignature rerandomization values.

TOB-NEARROBUST-4: Signature share linearization may slightly weaken robustness
Resolved in issue #199 and PR #319. Enforcing that exactly $N_1 = 2t + 1$ participants in presigning (introduced by the PR) eliminates these slight differences in behavior, and the marginal risk of denial of service is outweighed by efficiency gains.

TOB-NEARROBUST-5: Zero threshold causes integer overflow panic in debug mode
Resolved in PR #163. The relevant functions now use `checked_sub` to catch and propagate overflow errors instead of panicking.

TOB-NEARROBUST-6: Missing zeroization of presignature data
Resolved in PR #237. Both presignature structures, as well as structures related to OT-based ECDSA and key generation output, now derive the `ZeroizeOnDrop` trait.

TOB-NEARROBUST-7: Standard split-view attack can extract the secret key with 3t+2 signers
Resolved in PR #317 and PR #319. The Markdown files now document split-view attacks, and the presigning phase now requires that exactly $N_1 = 2t + 1$ parties participate in presigning.

TOB-NEARROBUST-8: Novel split-view attack can extract the secret key with 2t+3 signers
Resolved in PR #317 and PR #319. The Markdown files now document split-view attacks, and the presigning phase requires that $h \neq 0$ and exactly $N_1 = 2t + 1$ parties participate in presigning.

TOB-NEARROBUST-9: Inconsistent interpolation bounds between implementation and reference

Resolved in PR #295. The signing and presigning functions now accept the threshold as an argument and perform the suggested validation and interpolation.

TOB-NEARROBUST-10: Inconsistent threshold semantics across DKG and presigning protocols

Resolved in PR #310, PR #311, and PR #315. The implementation now uses distinct `MaxMalicious` and `ReconstructionLowerBound` types to distinguish semantic meaning.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to NEAR One under the terms of the project statement of work and has been made public at NEAR One's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.